

Clase 022 — Docker y contenedores para laboratorios de seguridad

Parte: **0 — Fundamentos y prerequisites** · Fuente: *Docker Documentation / NIST SP 800-190* 

Duración estimada: **110 min** · Nivel: **Fundamentos**

Objetivo

Usar contenedores para desplegar entornos vulnerables y herramientas de forma rápida, reproducible y desechable. Al terminar sabrás construir imágenes, ejecutar contenedores, orquestar con Docker Compose y comprender las nociones básicas de seguridad de contenedores, incluido su aislamiento respecto a una VM.

Resultados de aprendizaje

Al finalizar, el alumno podrá:

1. **Ejecutar** y gestionar contenedores con la CLI de Docker.
2. **Construir** imágenes con un Dockerfile.
3. **Orquestar** varios servicios con Docker Compose.
4. **Desplegar** laboratorios vulnerables (DVWA, Juice Shop) en contenedores.
5. **Explicar** el modelo de aislamiento y sus límites frente a una VM.

Temas

#	Tema	Por qué importa
1	Imágenes vs. contenedores	Plantilla vs. instancia
2	CLI de Docker	run, ps, exec, logs, stop
3	Dockerfile	Construir imágenes propias
4	Volúmenes y redes	Persistencia y conectividad
5	Docker Compose	Multi-servicio declarativo
6	Labs vulnerables	Entornos de práctica reproducibles
7	Aislamiento	Namespaces y cgroups
8	Seguridad de contenedores	Superficie y buenas prácticas

Definiciones y características

- **Imagen:** plantilla inmutable con el software y su entorno. Clave: se versiona por tags; base para crear contenedores.

- **Contenedor:** instancia en ejecución de una imagen. Clave: desechable, rápido de crear y destruir.
- **Dockerfile:** receta declarativa para construir una imagen. Clave: reproducibilidad y control de lo que se instala.
- **Volumen:** almacenamiento persistente fuera del ciclo de vida del contenedor. Clave: los datos no se pierden al recrear.
- **Namespaces/cgroups:** mecanismos del kernel que aíslan y limitan recursos. Clave: la base (más débil que una VM) del aislamiento de contenedores.
- **Docker Compose:** define varios servicios en un `compose.yml`. Clave: levanta un laboratorio completo con un comando.

Herramientas y preparación

Instala **Docker Engine** (o Docker Desktop). Verifica:

```
docker --version && docker run hello-world
```

Trabaja dentro de tu VM de laboratorio para no exponer contenedores vulnerables. Imágenes de práctica: `vulnerables/web-dvwa`, `bkimminich/juice-shop`. Ten Compose disponible (`docker compose version`).

Laboratorio guiado

1. Primer contenedor:

```
bash docker run -d --name web -p 8080:80 nginx docker ps ; curl -s localhost:8080 | head
```

2. Inspeccionar y entrar:

```
bash docker logs web ; docker exec -it web bash
```

3. Desplegar un lab vulnerable (solo en red aislada):

```
bash docker run -d -p 3000:3000 bkimminich/juice-shop
```

Abre `http://localhost:3000` desde la propia VM. 4. **Construir una imagen.** Crea un `Dockerfile` con una herramienta Python propia (p. ej. tu `pyscan.py`) y constrúyela:

```
dockerfile FROM python:3.12-slim COPY pyscan.py /app/pyscan.py ENTRYPOINT ["python",  
"/app/pyscan.py"]
```

```
bash docker build -t pyscan . && docker run --rm pyscan --help
```

5. Compose multi-servicio.

Escribe un `compose.yml` que levante DVWA + su base de datos y arráncalo con `docker compose up -d`.

6. Limpieza.

Detén y elimina lo creado; entiende que un contenedor es efímero:

```
bash docker rm -f web ; docker compose down
```

 **Nota ética y de seguridad:** las imágenes deliberadamente vulnerables (DVWA, Juice Shop) **jamás** deben exponerse a Internet ni a tu red doméstica. Ejecútalas solo en el laboratorio aislado y elimínalas

al terminar.

Ejercicios

1. Explica la diferencia entre una imagen y un contenedor con una analogía.
2. Mapea un volumen para que los datos de un contenedor persistan tras recrearlo.
3. Escribe un Dockerfile mínimo que empaquete una de tus herramientas Python.
4. Crea una red Docker interna y conecta dos contenedores que se comuniquen por nombre.
5. Compara aislamiento de contenedor vs. VM: ¿qué comparte cada uno con el host?
6. Investiga tres buenas prácticas de seguridad de imágenes (usuario no root, imagen mínima, escaneo).

Reto verificable

Crea con Docker Compose un laboratorio web vulnerable reproducible (por ejemplo DVWA con su base de datos) que se levante con un solo comando en tu red aislada, más un contenedor "atacante" con tus herramientas Python. Documenta cómo se lanza, cómo se conectan y cómo se destruye todo limpiamente.

Criterio de aceptación: `docker compose up -d` levanta la app vulnerable accesible solo desde la VM; el contenedor atacante alcanza a la víctima por la red interna de Compose; y `docker compose down` elimina todo sin dejar recursos huérfanos. Otro alumno reproduce el lab con tu `compose.yml`.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
<code>permission denied</code> al usar <code>docker</code>	Tu usuario no está en el grupo <code>docker</code> . Añádalo o usa <code>sudo</code> .
El puerto ya está en uso	Otro proceso ocupa el puerto host. Cambia el mapeo <code>-p</code> .
Los datos desaparecen al recrear	No usaste volumen. Monta un volumen para persistencia.
Lab vulnerable accesible desde fuera	Bindeaste a <code>0.0.0.0</code> en red no aislada. Restringe a la VM/red interna.
Imagen enorme/lenta	Base pesada y capas mal ordenadas. Usa imágenes <code>slim</code> / <code>alpine</code> y aprovecha la caché.

Preguntas frecuentes

? ¿Un contenedor es tan seguro como una VM? No. Comparte el kernel del host, por lo que su aislamiento es más débil. Para malware o exploits de kernel, usa VMs; para servicios y labs web reproducibles, contenedores.

? ¿Por qué Docker para labs si ya tengo VMs? Rapidez y reproducibilidad: levantas y destruyes entornos en segundos y compartes un `compose.yml`. Se complementan con las VMs de la Clase 004.

? **¿Debo actualizar/escanear mis imágenes?** Sí: las imágenes contienen dependencias con CVEs.

Escanéalas (Trivy, Grype) y usa bases mínimas y actualizadas.

? **¿Contenedor como root es peligroso?** Sí: si el contenedor se compromete, el proceso corre como root y un escape sería más dañino. Usa usuarios no privilegiados en tus imágenes.

Referencias

- Docker Documentation — <https://docs.docker.com/>
- NIST SP 800-190, *Application Container Security Guide* — <https://csrc.nist.gov/pubs/sp/800/190/final>
- OWASP Juice Shop — <https://owasp.org/www-project-juice-shop/>
- CIS Docker Benchmark — <https://www.cisecurity.org/benchmark/docker>

Siguiente clase

Clase 023 - Sistemas operativos: procesos, memoria y syscalls